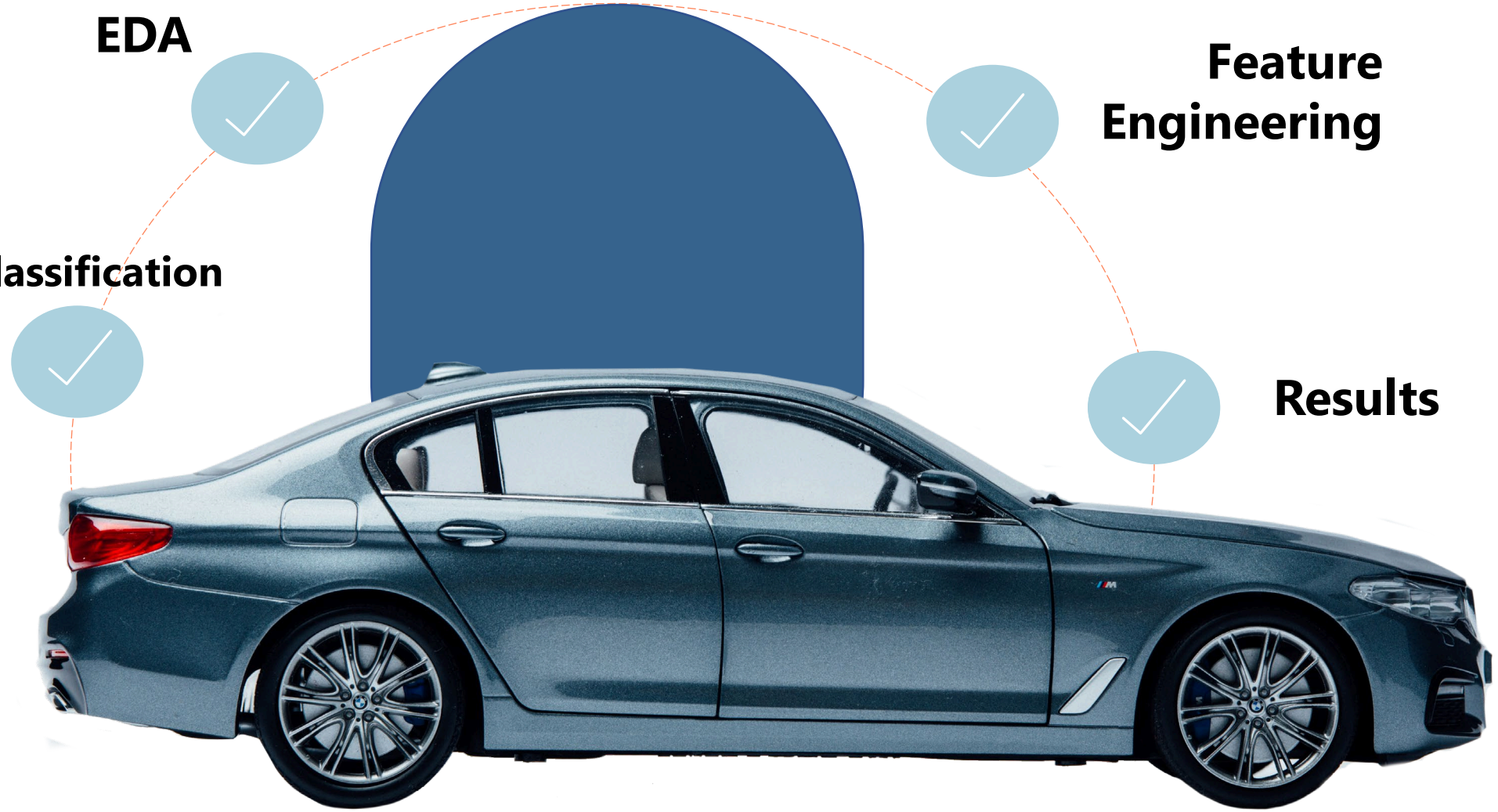
Insurance Policy Approval: Combining Risk Probability and Potential Loss Prediction

Christopher Tsialas
Matthew Hawkes

Meet the Team

Table of Contents



EDA

Feature Engineering

Datasets: Regression/Classification

Results

Why Two Datasets?

Insurance risk has two parts:

1. Will a claim happen? (probability, yes/no) (Classification)

2. If it happens, how expensive will it be? (severity, dollar loss) (Regression)

Dataset 1: Porto Seguro Safe Driver Prediction (Classification)

- Predict whether a driver will file a claim
- Each row is a driver/policy profile with anonymized features
- Target is binary: claim filed (1) vs no claim (0)
- Main challenge: claims are rare, so we evaluate with probability-based metrics and focus on catching positives

Dataset 2: AllState Claims Severity (Regression)

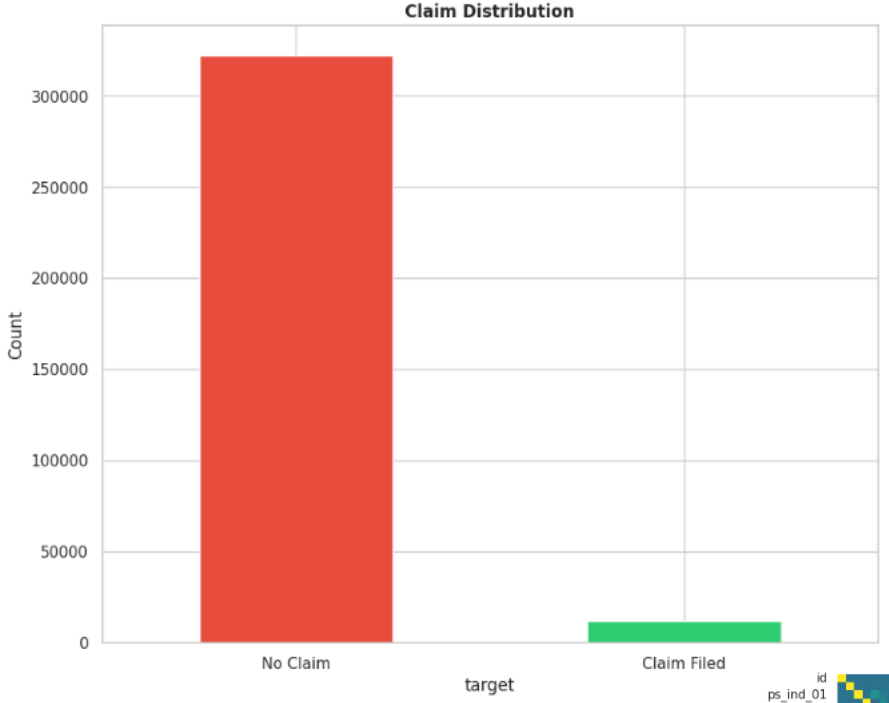
- Predict the expected cost of a claim if it occurs
- Each row is a claim case with anonymized features
- Target is continuous: loss amount
- Main challenge: loss is skewed (a few very large claims), so we care about error size, not just “direction”

Problem framing and Success Metric

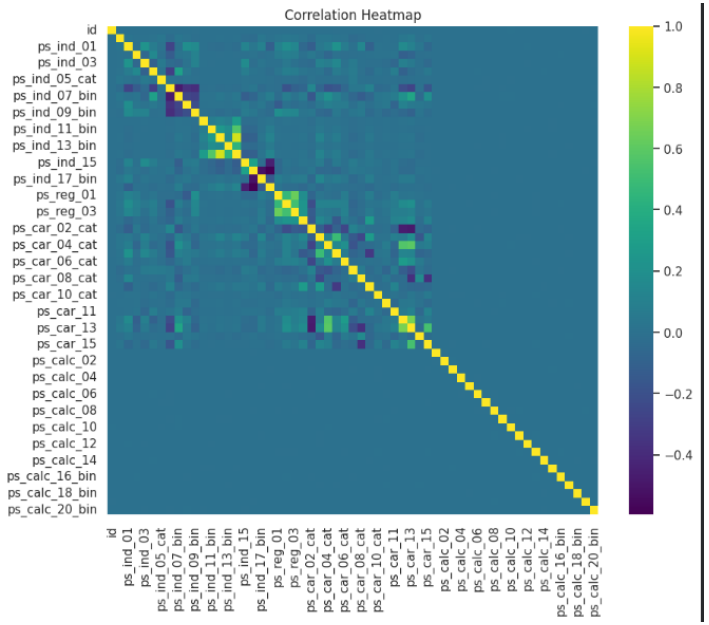
- Predict probability of a claim for each driver (target 0 vs 1)
- Predict loss amount for claims dataset (continuous “loss”)
- Use metrics that match the business risk of missing claims
- Translate performance into a decision idea: approve, price higher, or reject

Safe Driver EDA: Claim Imbalance + Missingness

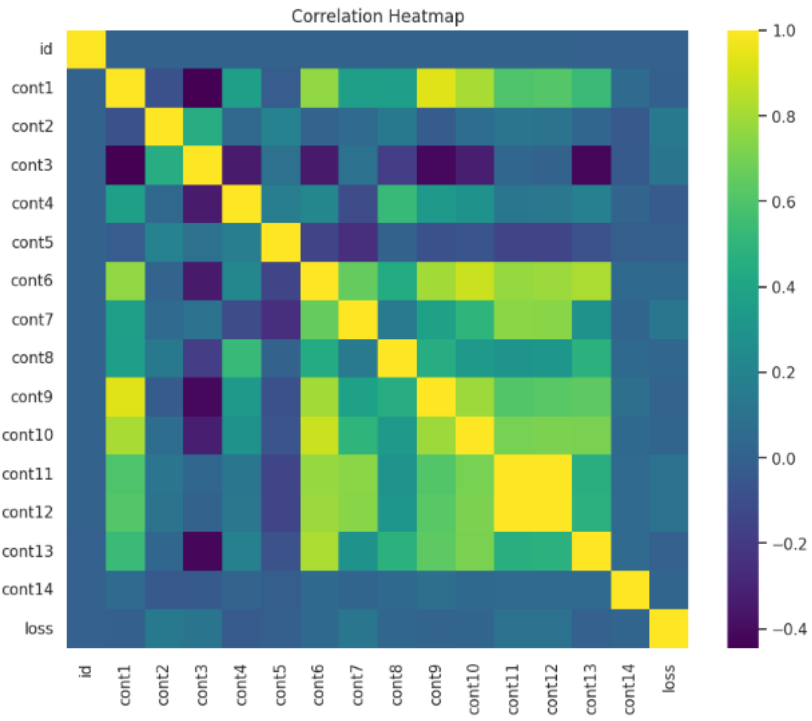
- Claims are rare, so the target is strongly imbalanced and accuracy alone would be misleading.
- The correlation heatmap shows mostly weak relationships, meaning the signal is spread across many features, not one obvious predictor.
- MMissingness is concentrated in a handful of columns, so targeted imputation makes more sense than dropping data.
- These patterns justify using PR-focused evaluation and recall-friendly thresholding in the modeling step.

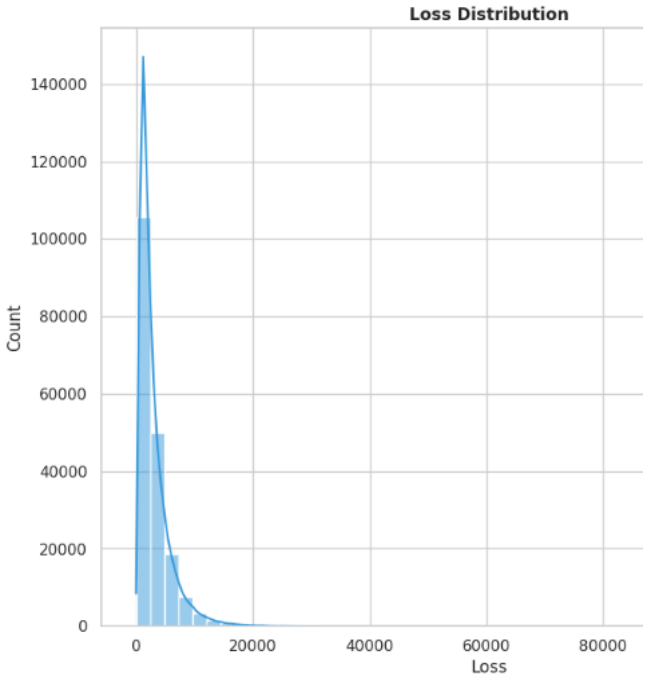



```
No Claim: 322484 (96.36%)
Claim: 12176 (3.64%)
```



Claims Severity EDA: Loss Distribution






```
Loss summary:
count    188318.000000
mean       3037.337686
std        2904.086186
min           0.670000
25%        1204.460000
50%        2115.570000
75%        3864.045000
max      121012.250000
```

- Loss is heavily right-skewed: lots of small claims, few huge ones drive risk.
- Mean isn't representative, so MAE matters for "typical" error.
- Big claims can dominate RMSE, so track both MAE and RMSE.
- Most feature correlations are weak, so non-linear models help.
- Missingness is concentrated in a few columns, so pipeline imputation works well.

Feature engineering: Classification

```
Binary Features by Signed Claim-Rate Difference (%):
ps_ind_10_bin              +2.24%
ps_ind_17_bin              +2.14%
ps_ind_13_bin              +1.74%
ps_ind_12_bin              +1.59%
ps_ind_07_bin              +1.46%
ps_ind_06_bin              -1.29%
ps_ind_11_bin              +1.19%
ps_ind_16_bin              -1.09%
ps_ind_08_bin              +0.67%
ps_ind_09_bin              -0.42%
ps_ind_18_bin              +0.22%
ps_calc_20_bin             -0.04%
ps_calc_16_bin             +0.04%
ps_calc_15_bin             -0.03%
ps_calc_18_bin             +0.03%
ps_calc_19_bin             -0.03%
Categorical Features by Signed Claim-Rate Difference (%):
ps_ind_04_cat              +35.80%
ps_car_01_cat              +27.13%
ps_ind_02_cat              +14.84%
ps_car_04_cat              +5.45%
ps_car_09_cat              +5.34%
```

- Claim-Rate Difference for each feature of the dataset.
- This metric determines how important each feature is to the prediction, either decreasing or increasing risk.
- + values = increase risk
- - values = decrease risk
- Way to understand feature impact without knowing the semantics of the column.

Feature engineering: Regression

```
Categorical Features by Signed Mean Loss Difference:
cat116                    48345.91
cat105                    48345.91
cat113                    36980.04
cat115                    30987.41
cat89                     15609.24
cat102                    15609.24
cat57                     7198.59
cat114                    6022.78
cat101                    5944.64
cat110                    5601.48
cat111                    5442.07
cat7                      5177.07
cat106                    5067.90
cat90                     4432.64
cat103                    4265.61
cat92                     4165.93
cat64                     3490.79
```

- Mean Loss Difference for each feature of the dataset
- This metric determines how important each feature is to the prediction, either decreasing or increasing loss.
- + values = increase loss
- - values = decrease loss
- Showing how much a feature on average impacts the loss

Feature engineering: RiskSum idea

```
Average RiskSum_total by Target:
Target 0: 0.0640
Target 1: 0.0860
```

```
Average RiskSum_total by Loss Quartile:
Quartile 0: 2886.9199
Quartile 1: 2923.4520
Quartile 2: 2929.2334
Quartile 3: 3064.6299
```

- Build “RiskSum” by combining signed claim-rate or loss differences across feature types
- Create flags like RiskFlag / HighRiskFlag for a simple rule-based comparator
- Use this to explain what “high risk” looks like, even with anonymized columns

Preprocessing Pipeline

```python
cols_to_drop = ['id']
driver_train_df_model = driver_train_df.drop(columns=cols_to_drop+['target'])
driver_test_df_model  = driver_test_df.drop(columns=cols_to_drop)

claims_train_df_model = claims_train_df.drop(columns=cols_to_drop+['loss'])
claims_test_df_model  = claims_test_df.drop(columns=cols_to_drop)

y_driver_train = driver_train_df['target']
y_claims_train = claims_train_df['loss']


def build_preprocessor(numeric_features, categorical_features):
    numeric_transformer = Pipeline(steps=[
        ('imputer', SimpleImputer(strategy='median')),
        ('scaler', StandardScaler())
    ])

    categorical_transformer = Pipeline(steps=[
        ('imputer', SimpleImputer(strategy='most_frequent')),
        ('onehot', OneHotEncoder(handle_unknown='ignore'))
    ])

    return ColumnTransformer(
        transformers=[
            ('num', numeric_transformer, numeric_features),
            ('cat', categorical_transformer, categorical_features)
        ]
    )

driver_preprocessor = build_preprocessor(driver_bin_cols+driver_cat_cols+driver_cont_cols, [])
claims_preprocessor = build_preprocessor(claims_cont_cols, claims_cat_cols)
```

- Numeric: median imputation + standardization
- Categorical (claims dataset): most-frequent imputation + one-hot encoding
- Split: train/validation, and stratified split for the imbalanced classification dataset

Models Compared and Why

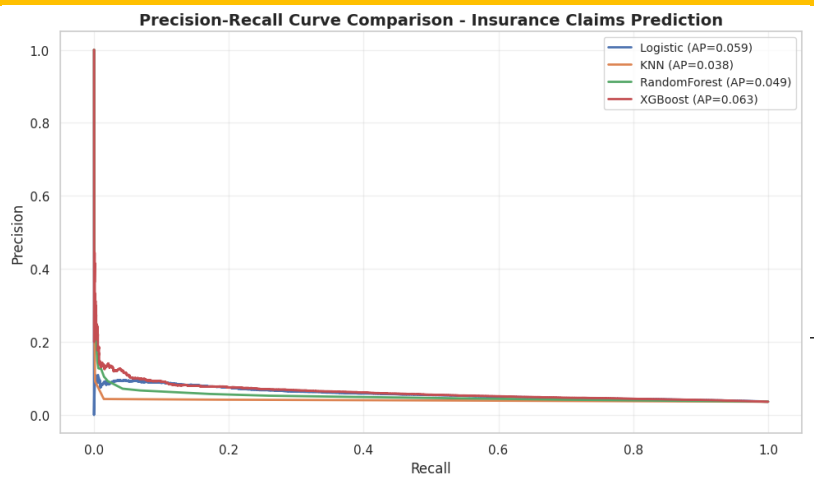
	Model	Threshold(F2)	Best F2	PR AUC	ROC AUC	Accuracy	Precision	Recall	F1-score
3	XGBoost	0.476025	0.195361	0.060431	0.629055	0.634674	0.054474	0.552772	0.099175
2	RandomForest	0.482008	0.196555	0.060093	0.630531	0.611187	0.053796	0.583984	0.098517
0	Logistic	0.498865	0.196034	0.059213	0.628499	0.621691	0.054088	0.570021	0.098801
1	KNN	0.000000	0.158793	0.041407	0.533318	0.036380	0.036380	1.000000	0.070206

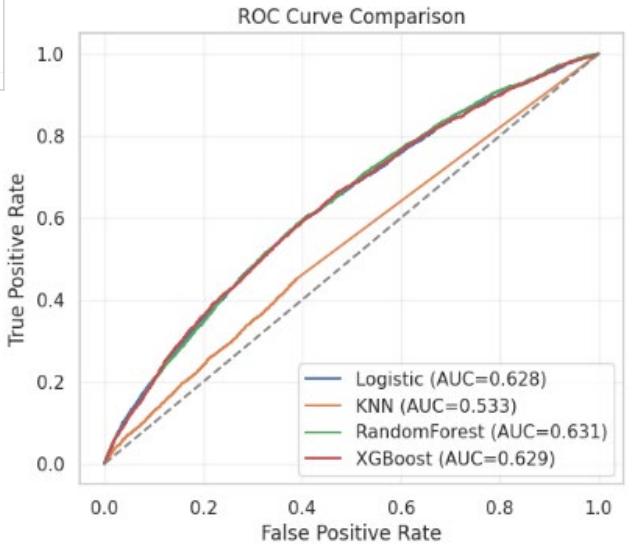
Safe Driver models tested

- Logistic Regression (balanced class weights)
- KNN
- Random Forest (balanced subsample)
- XGBoost (aucpr eval metric, scale_pos_weight from train split)

The large dataset with an extremely imbalanced dataset made initial models not perform amazingly.

Classification Evaluation: Thresholding + Curves





Evaluation choices

- Predict probabilities, then pick a decision threshold that maximizes F2 (recall-weighted)
- Report ROC AUC because it measures how well the model ranks positives and negatives, independent of extreme class imbalance

Regression Evaluation: Metrics + Model Pick

	Model	RMSE	MAE	R2
3	XGBoost	3.431756e+06	1178.702677	0.579421
1	Ridge	4.012258e+06	1290.538441	0.508278
2	RandomForest	4.034452e+06	1290.559387	0.505558
0	Linear	4.066705e+06	1291.425777	0.501605

Claims Severity models tested

- Linear Regression
- Ridge
- Random Forest Regressor
- XGBoost Regressor

Metrics reported

- MAE and R^2 Plus RMSE

Final Model Creation: Classification

```python
    'model__colsample_bytree': [0.6, 0.8, 1.0],
    'model__min_child_weight': [1, 3, 5]
}

# Cross-validation
cv = StratifiedKFold(
    n_splits=3,
    shuffle=True,
    random_state=RANDOM_STATE
)

# RandomizedSearchCV
rand_search = RandomizedSearchCV(
    estimator=xgb_pipe,
    param_distributions=param_dist,
    n_iter=20,
    scoring='average_precision',
    cv=cv,
    n_jobs=-1,
    random_state=RANDOM_STATE,
    verbose=1
)

# Fit model
```

Tuned XGBClassifier using:

- RandomizedSearchCV which provides good results while not undergoing exhaustive GridSearch
- StratifiedKFold which ensures proportional imbalance in each fold from the dataset
- Average_precision, which better handles class imbalance than other metrics, like accuracy

Final Model Creation: Regression

```
    'model__min_child_weight': [1, 3, 5]
}

# Cross-validation
cv = KFold(
    n_splits=3,
    shuffle=True,
    random_state=RANDOM_STATE
)

# RandomizedSearchCV
rand_search = RandomizedSearchCV(
    estimator=xgb_pipe,
    param_distributions=param_dist,
    n_iter=20,
    scoring='neg_root_mean_squared_error',
    cv=cv,
    n_jobs=-1,
    random_state=RANDOM_STATE,
    verbose=1
)

# Fit model
rand_search.fit(claims_X_train, claims_y_train)
```

Tuned XGBRegressor using:

- RandomizedSearchCV which provides good results while not undergoing exhaustive GridSearch
- KFold, as StratifiedKFold is unnecessary for continuous data
- Neg_root_mean_squared_error, which is better suited for continuous data.

Limitations and Next Steps

- Anonymized features limit interpretation, so use feature importance and segment error checks.
- Imbalanced claims make thresholds fragile, so calibrate probabilities and stress-test cutoffs.
- Loss is heavy-tailed, so try log-loss or robust objectives and track MAE + RMSE.
- Turn outputs into a rule: expected cost and cutoffs for approve vs reject.

Final Takeaway: How This Becomes Policy Approval

- Claim model estimates probability of claim
- Severity model estimates expected loss
- Combine into expected cost: P(claim) × E(loss | claim proxy)
- Use a cutoff policy depending on risk tolerance (reject, approve, approve with pricing)